

EE 446 Lab 6 Report - Aarav Wadhwani

Required **Part 1** Content in Lab6_Report.pdf

Edge Impulse Project: <https://studio.edgeimpulse.com/public/989437/live>

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw inputs consist of 3-axis accelerometer data sampled at 100 Hz. The data is processed in a 2,000 ms window with a window increase (stride) of 220 ms. The specific axes used are:

- accX
- accY
- accZ

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

- **Type of features:** Spectral features (generated by the Spectral Analysis processing block).
- **Number of input features:** 33 features.
- **Actual feature names** (5 examples):
 - accX RMS
 - accX Peak 1 Height
 - accY Peak 3 Height
 - accX Peak 1 Freq
 - accY RMS

3. NN classifier outputs: State the outputs of the NN classifier. EE 446 TinyML | Lab 6 Submission Guidelines Submit the public GitHub Lab 6 folder link through Canvas.

The NN classifier outputs two nominal classes:

- **nominal** (representing the fan/device operating normally)
- **off** (representing the fan/device being powered down)

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

- **Type of features:** Spectral features (a selected subset of the axes processed by the K-means block).
- **Number of features used:** 4 features.
- **Feature names:**
 - accX RMS
 - accX Peak 1 Height
 - accY Spectral Power 2.0 - 5.0 Hz

- accZ Spectral Power 2.0 - 5.0 Hz

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 614 us, anomaly 526 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.402344
  off: 0.597656
Anomaly prediction: 15.944424
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 614 us, anomaly 518 us, postprocessing 49 us
#Classification predictions:
  nominal: 0.003906
  off: 0.996094
Anomaly prediction: 1000.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 49 ms, inference 611 us, anomaly 560 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.996094
  off: 0.000000
Anomaly prediction: 6.209517
Starting inferencing in 2 seconds...
```

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

The deployment results show the model successfully running on-device. In the third inference cycle shown, the model correctly identifies the **"nominal"** state with a high confidence score of **0.996**.

Reliability Discussion: The classifier output should **not** always be trusted. While a Neural Network is excellent at mapping inputs to known labels, it is "greedy" and will try to force any input into one of its pre-defined categories.

- **When it is reliable:** When the input data closely matches the training distribution (i.e., when the Anomaly score is low).
- **When it is not reliable:** If the device experiences a state it wasn't trained for (like a mechanical failure or external vibration), the NN might still report high confidence for "nominal" or "off." This is evidenced in your second inference result, where the NN

predicted "off" (0.99), but the **Anomaly prediction was 1000.00**. This high anomaly score serves as a "safety check," telling us that the classifier's result should be ignored because the input is completely unlike the training data.

Part 2:

Answer the following briefly:

- What threshold did you choose for anomaly detection, and why?

I used the 95th percentile () of the training normal reconstruction errors as the threshold, so most normal windows fall below it while leaving headroom to flag higher-error windows as anomalies.

- *How does reconstruction error help distinguish normal and anomalous activity?*

Reconstruction error measures how well the autoencoder can rebuild a window; normal patterns reconstruct well (low error), while unusual patterns reconstruct poorly (high error).

- *What information from the Python notebook must be preserved when deploying the model to Arduino?*

Keep the trained int8 model, the chosen threshold, and the input scaling/quantization details (input/output scale and zero point) that the TFLite model uses.

- *Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?*

The Arduino sketch must use the same preprocessing (window length, axis order, flattening, scaling) so the model sees the same input distribution it was trained on.

- *What are the main limitations of this anomaly detection method when deployed on a microcontroller?*

Main limits on a microcontroller: limited memory/compute, fixed threshold sensitivity, possible drift in real-world data, and reduced precision from int8 quantization.